CIS-120 UEBA Insider Threat Detection: Kiro Build Plan

Oct 17, 2026 · @mayank

Project brief

CIS-120 builds a User and Entity Behaviour Analytics (UEBA) system that learns each employee's normal activity from logs and ranks users by how far their behaviour departs from it. The HOD requires three things for every PS code: a running demo, the code, and architecture documentation.

- **Input:** the CMU/SEI CERT Insider Threat Test Dataset, version r4.2 (logon, device, file, http and email logs).
- **Output:** a dashboard listing the riskiest users, with the reasons behind each score.
- **Evidence:** precision, recall and false-positive rate measured against the labelled insiders in the dataset.
- **Deliverables:** a working app, a Git repo with README and tests, and an architecture document with diagrams.
- **Design rule:** keep the models simple and explainable. Put the effort into features, because a literature review of 131 CERT studies found feature engineering matters more than model complexity.

How to use this doc with Kiro

Give Kiro the prompt in the last section, then let it work one phase at a time and check each phase against its acceptance test before moving on.

1. Download the CERT r4.2 dataset yourself (see References) and unpack it into a `data/raw/` folder. Kiro cannot fetch it for you.
2. Create an empty project folder and open it in Kiro.
3. Paste the main prompt from the last section into Kiro's spec mode. Kiro should produce requirements, design and task lists. Read them and fix anything that disagrees with this doc (menu names may differ by Kiro version).
4. Save the Dataset, Architecture and Feature sections as a steering or context file in the project so Kiro keeps them in view.

5. Run the phases in order. After each one, run the acceptance check from the phase table. Do not start the next phase until it passes.
6. Use the follow-up prompts in the last section whenever Kiro drifts or a phase fails.

References and resources

Twelve sources cover the data, the method and the architecture. They were found through web search on 2026-10-17 and read as search excerpts, so open each one before you cite it in your report.

Source	What it gives you	How to use it
SEI Insider Threat Test Dataset (doi 10.1184/R1/12841247.v1)	The dataset itself, from CMU's Software Engineering Institute	Download r4.2 and cite this DOI in your report
Deep Learning for Insider Threat Detection: Review	Describes the CERT log files (logon, email, http, file, device, psychometric) and the r4.2 vs r6.2 difference	Use for the dataset section and the file descriptions
Predicting Malicious Insider Threat Scenarios	The three r4.2 attack scenarios, 30 malicious users each	Use to design scenario-specific features and to explain your labels
Insider Detection using Deep Autoencoder and VAE	r4.2 covers 1,000 simulated employees, 01 Jan 2010 to 31 May 2011	Use for the time range when you split train and test
Systematic review of ML insider threat detection on CERT	Finds r4.2 results are inflated and feature engineering beats model complexity	Cite in your limitations section
A Machine Learning Based Framework for User-Centered Insider Threat Detection (Le, PhD thesis)	Anomaly detection with Isolation Forest, ensembles of detectors, feature extraction on CERT	Use as the main methods reference
Insider Threat Detection using Ensemble and Sequential Models (NCI thesis)	Compares Random Forest, Isolation Forest, LSTM, GRU and stacking	Use for the model comparison discussion

Source	What it gives you	How to use it
Mask-Based Window-Level Insider Threat Detection	Unsupervised, window-level UEBA on r4.2	Optional stretch: score time windows, not only days
What is UEBA? (Cyberhaven)	The loop: ingest, baseline, detect anomalies, score risk	Basis for the architecture layers
Logpoint UEBA overview	Commercial UEBA needs at least 30 days of logs to set a baseline	Justify your baseline window
Splunk UEBA dashboards	Shows feature value next to its baseline threshold	Copy the idea for your explanation panel
Insider Threat Detection Engine (GitHub, Raoof128/ITDE)	Layout with separate ingestion and analytics folders, weighted risk scoring, an architecture doc	Study the structure only; write your own code

Dataset plan

Use CERT r4.2 and work on aggregated daily features, not raw rows, because the full dataset is roughly 20GB of logs for 1,000 employees. r4.2 has three insider scenarios with 30 malicious users each, so about 90 labelled insiders.

File	Records	Used for
logon.csv	Logon and logoff events	After-hours and weekend activity, number of machines used
device.csv	Removable drive connect and disconnect	USB usage, especially after hours
file.csv	File activity involving removable media	Copies to removable drives

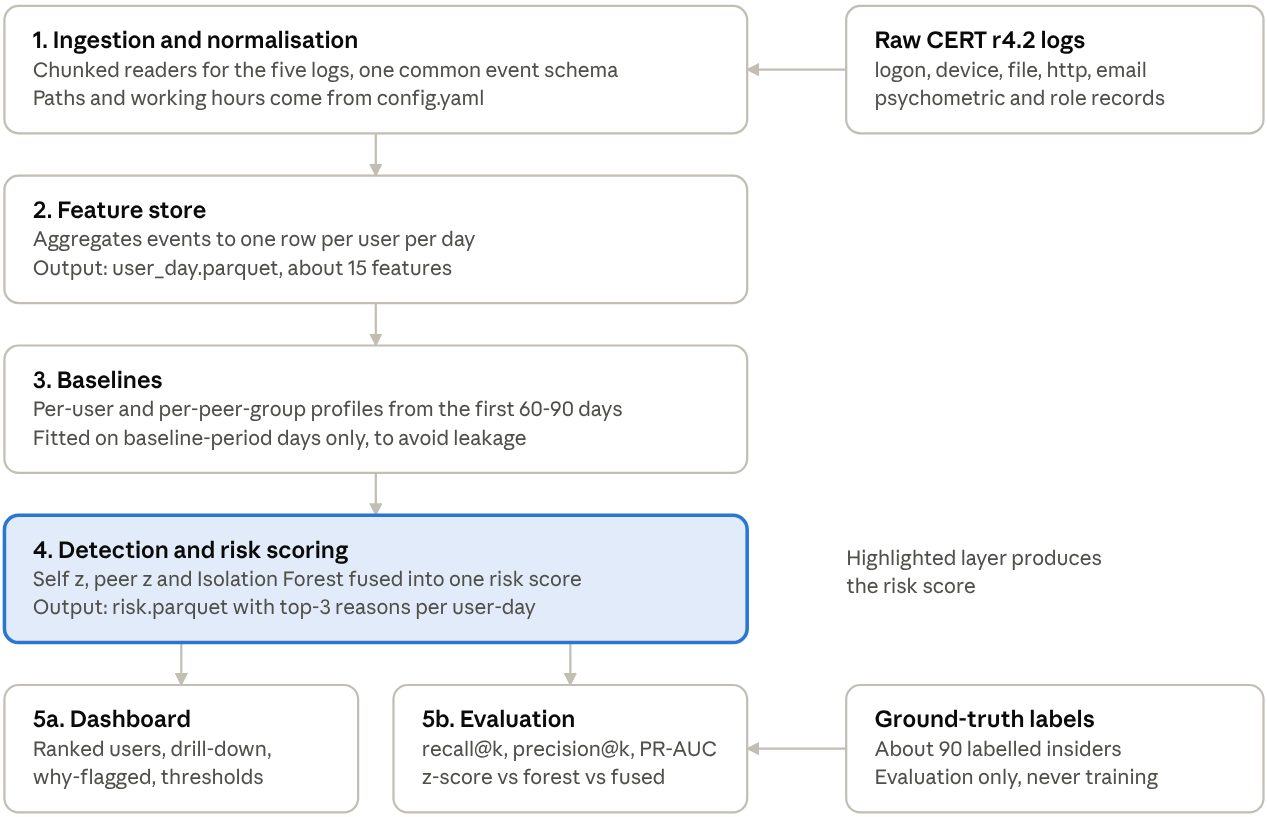
File	Records	Used for
http.csv	Web visits, downloads, uploads	Job sites, upload sites, volume
email.csv	Email sent and received	External recipients, attachments
psychometric.csv and monthly employee records	Personality scores and role or team	Peer groups (same role)
Ground-truth answer files shipped with the dataset	Which users are insiders and when	Evaluation only, never for training

Rules for Kiro to follow

- Never hard-code column names. First run a profiling script that prints each file's header, row count and date range, then write the parsers from that output. The headers differ between dataset versions.
- Read files in chunks (pandas `chunksize` or DuckDB) and write a compact Parquet table with one row per user per day.
- Build a working subset first: all labelled insiders plus about 300 randomly chosen normal users. Scale up only after the subset pipeline passes its tests.
- Split by time, not at random. Use the first 60 to 90 days of each user as the baseline period (the minimum is 30), and score every later day. A random split would leak future behaviour into the baseline.
- Keep `data/` out of Git and read its location from `config.yaml`.
- Build a small synthetic log generator with the same schema so the demo and the tests run without the 20GB download.

Architecture

Raw logs pass through five layers to a ranked, explained dashboard



system architecture · 5 layers, 2 inputs

Logs flow down through ingestion, features and baselines into scoring, which also reads the daily features directly. Scoring feeds both the dashboard and the evaluation harness, and the ground-truth labels enter only at evaluation.

Folder structure

```
cis120-ueba/
  config.yaml        paths, working hours, site lists, score weights
  requirements.txt
  README.md
  scripts/           profile_data.py, synthetic log generator
  src/ueba/          ingestion, features, baselines, models, scoring,
evaluation
  app.py             Streamlit dashboard
  tests/             unit tests per phase
  docs/              ARCHITECTURE.md, DEMO.md
  reports/           eval_report.md and plots
  data/              raw and processed files (kept out of Git)
```

Tech stack

- Python 3.11 with pandas and DuckDB or pyarrow for chunked processing
- scikit-learn for Isolation Forest
- Streamlit and Plotly for the dashboard
- pytest for tests, PyYAML for configuration

Feature and scoring design

Each user-day gets about 15 features, three independent scores, and one fused risk score with the top reasons attached.

Feature (per user per day)	Source	Why it matters
Logons, after-hours logons, weekend logons	logon	Scenario 1: starts working late
Distinct machines logged into	logon	Using other people's machines
USB connects, after-hours USB connects	device	Scenarios 1 and 3: data staging
Files copied to removable media	file	Data exfiltration
Visits to job and recruitment sites	http	Scenario 2: leaving for a competitor
Visits to file-upload and leak sites, uploads	http	Scenario 1: uploading to leak sites
Distinct domains, total web requests	http	General activity level

Feature (per user per day)	Source	Why it matters
Emails sent, external recipients, attachments	email	Data leaving the company

Three scores per user-day, each scaled to 0 to 1

1. **Self z-score:** how far each feature is from that user's own baseline mean, measured in standard deviations, capped at 5. Take the mean of the three largest values.
2. **Peer z-score:** the same calculation against the user's role group, to catch users who were always odd.
3. **Isolation Forest score:** an unsupervised model trained only on baseline-period days, giving a 0 to 1 anomaly score for each later day.

```
R_{u,d} = 100 \times (0.5\,S_{self} + 0.3\,S_{iso} + 0.2\,S_{peer})
```

A user's weekly risk is the sum of their last 7 daily scores, so a slow build-up ranks above a single noisy day. The weights are starting values: tune them on a validation set of users and record the final values in the architecture document.

Explainability is required. For every flagged user-day store the top three contributing features with their value, baseline mean and z-score. The dashboard shows this as "why flagged", like the baseline-vs-value panels in commercial UEBA tools.

Phased build plan

Seven phases, each ending in something you can run and an acceptance check you can verify. A phase is done only when its check passes.

Phase	Tasks	Deliverable	Acceptance check
0. Setup and profiling	Create repo, `config.yaml`, virtual environment, `requirements.txt`; write the profiling script; write the synthetic log generator	Project skeleton and a profile report of every CERT file	`pytest` runs; profiling prints header, row count and date range for each file

Phase	Tasks	Deliverable	Acceptance check
1. Ingestion	Chunked parsers for the five logs; normalise to one event schema; aggregate to a user-day Parquet table	`user_day.parquet` with the features from the Feature section	Row count equals users times active days; no null user IDs; unit tests pass on synthetic data
2. Baselines and statistical scoring	Per-user rolling baseline; per-peer-group baseline; self and peer z-scores with capping	`scores_stat.parquet`	A hand-built user with a planted USB spike gets a z-score above 4 on that day in a test
3. Isolation Forest and fusion	Train on baseline-period days only; score later days; fuse into daily and 7-day risk; store top-3 reasons	`risk.parquet` with reasons	Trained model is saved and reloaded in a test; no training row is dated after the baseline cutoff
4. Evaluation harness	Join risk to ground-truth labels; compute recall@k, precision@k, PR-AUC; compare z-score only, Isolation Forest only and fused	`eval_report.md` and plots	Report runs from one command and lists all three variants side by side
5. Dashboard	Streamlit app: ranked users, user drill-down with timeline, why-flagged panel, date and threshold controls	`app.py`	Opens in under 10 seconds on the subset; clicking a known insider shows their top reasons

Phase	Tasks	Deliverable	Acceptance check
6. Docs and demo polish	README, architecture document with diagrams, demo script, screenshots, final tests	`docs/ARCHITECTURE.md`, `docs/DEMO.md`	A new person can clone the repo, follow the README and see the dashboard on synthetic data

Evaluation and demo plan

Report detection quality with numbers against the labelled insiders, and rehearse a five-minute demo that ends on one convincing case.

Metrics to report

- Recall@20 and recall@50 at user level: how many of the ~90 insiders appear in the top 20 and top 50 users.
- Precision@20 at user level.
- PR-AUC at user-day level. Insider days are rare, so plain accuracy is misleading and should not be reported.
- Alerts per day at your chosen threshold, to show analyst workload.
- A three-way comparison: z-score only, Isolation Forest only, fused score.
- A limitations paragraph: r4.2 is synthetic and results will look better than on real data.

Demo script (5 minutes)

1. Show the architecture diagram in one sentence per layer.
2. Open the dashboard on the ranked user list.
3. Click a known insider and show the timeline and the why-flagged panel.
4. Move the threshold slider and show how alerts per day change.
5. Show the evaluation table and state the limitations.

Architecture documentation checklist

- [ ] System context and layer diagram
- [ ] Data flow diagram from raw logs to dashboard

- [ ] Feature table with source file and rationale
- [ ] Scoring formula and final weights
- [ ] Evaluation results and limitations
- [ ] Folder structure and how to run each phase
- [ ] References, with the SEI dataset DOI

Risks and fallbacks

The two biggest risks are dataset size and weak detection results, and both have a planned fallback.

Risk	Likelihood	Fallback
20GB dataset will not load or download in time	High	Use the subset plan, or the synthetic generator, and say so in the report
Column names differ from what Kiro assumed	Medium	Profiling script in Phase 0; parsers are written from its output
Isolation Forest finds few insiders	Medium	Report z-score results as the main method and treat Isolation Forest as a comparison
Time runs out before the dashboard is done	Medium	Phase 5 can start from a simple ranked table plus one chart; polish later
Results look too good to be true	Medium	Check for leakage first (baseline cutoff, labels used in features), then state the r4.2 limitation
Kiro generates code you cannot explain	High	Read every module before moving on; ask Kiro to add comments and a one-paragraph explanation per module

The Kiro prompt

Paste the main prompt first, then use the follow-up prompts one phase at a time.

Main prompt

```
You are helping me build CIS-120: Insider Threat Detection Using User and
Entity Behaviour Analytics (UEBA). This is a university project. I must
present a running demo, the code, and architecture documentation. I need to
be able to explain every part, so keep the code simple, commented and
modular.

GOAL
Build a UEBA system on the CMU/SEI CERT Insider Threat Test Dataset r4.2. It
learns each user's normal behaviour, scores later days against that baseline,
ranks users by risk, explains why each user was flagged, and measures
detection quality against the dataset's ground-truth labels.

TECH STACK
Python 3.11, pandas and DuckDB or pyarrow for chunked processing, scikit-
learn (IsolationForest), Streamlit and Plotly for the dashboard, pytest for
tests, PyYAML for config. No cloud services. No network calls at runtime.

DATA RULES
- Raw data lives in data/raw/ and is never committed. The path comes from
config.yaml.
- Do NOT assume column names. First write scripts/profile_data.py that prints
each CSV's header, row count and date range. Write the parsers from that
output.
- The dataset is about 20GB. Read in chunks and aggregate to one row per user
per day, saved as Parquet.
- Also write a synthetic log generator with the same schema, so tests and the
demo run without the full dataset.
- Work on a subset first: all labelled insiders plus about 300 random normal
users.
- Split by time. The first 60-90 days per user are the baseline period
(minimum 30). Models and baselines are fitted only on baseline-period data.
Never use ground-truth labels for training or features, only for evaluation.

FEATURES (per user per day)
logon count, after-hours logons, weekend logons, distinct machines; USB
connects, after-hours USB connects; files copied to removable media; visits
to job sites; visits and uploads to file-upload or leak sites; distinct
domains; total web requests; emails sent, external recipients, attachments.
Make working hours and the site category lists configurable in config.yaml.

SCORING
1. Self z-score vs the user's own baseline, capped at 5. Score = mean of the
top 3 feature z-scores, scaled 0-1.
```

```
2. Peer z-score vs the user's role group, same method.
3. IsolationForest trained on baseline-period user-days, anomaly score scaled
0-1.
Daily risk = 100 * (0.5*self + 0.3*isolation + 0.2*peer). Weekly risk = sum
of the last 7 daily risks. Weights are in config.yaml. For every user-day
store the top 3 contributing features with value, baseline mean and z-score.

EVALUATION
Compute recall@20, recall@50, precision@20 at user level, and PR-AUC at user-
day level. Compare z-score only, IsolationForest only, and fused. Write
results to reports/eval_report.md with plots. Do not report plain accuracy.

DASHBOARD (Streamlit)
Ranked list of users by weekly risk; date and threshold controls; user drill-
down with a timeline of daily risk; a why-flagged panel showing feature value
vs baseline; an alerts-per-day view.

SUGGESTED STRUCTURE
config.yaml, requirements.txt, README.md, scripts/,
src/ueba/{ingestion,features,baselines,models,scoring,evaluation}/, app.py,
tests/, docs/ARCHITECTURE.md, docs/DEMO.md, reports/.

WORKING STYLE
- Start in spec mode: produce requirements, design and tasks, then wait for
my approval.
- Work in 7 phases: (0) setup and profiling, (1) ingestion, (2) baselines and
statistical scoring, (3) isolation forest and fusion, (4) evaluation, (5)
dashboard, (6) docs and demo polish. Finish and test one phase before
starting the next.
- Write unit tests for each phase. Run them and show me the results.
- If something is unclear, or a file does not match what you expected, stop
and ask me instead of guessing.
- After each module, add a short plain-English explanation of what it does
and why, so I can explain it in my viva.
- Finally write docs/ARCHITECTURE.md with diagrams in Mermaid, the feature
table, the scoring formula, the final weights, the evaluation results and the
limitations.

START NOW with Phase 0 only.
```

Follow-up prompts

PHASE 1: Implement ingestion for the five log files using the profile output. Produce user_day.parquet. Add tests on synthetic data. Show me row counts and 5 sample rows.

PHASE 2: Implement per-user and per-peer-group baselines and z-score scoring. Add a test with a planted USB spike that must score above 4. Show me the top 10 scored user-days.

PHASE 3: Train the IsolationForest on baseline-period days only, fuse the three scores, store the top 3 reasons per user-day. Add a test proving no training row is dated after the baseline cutoff.

PHASE 4: Build the evaluation harness. Run it on the subset and show me the three-way comparison table. Check for label leakage before showing results.

PHASE 5: Build the Streamlit dashboard with ranked users, drill-down, why-flagged panel and threshold controls. Make it run on the synthetic data with one command.

PHASE 6: Write README.md, docs/ARCHITECTURE.md (Mermaid diagrams), docs/DEMO.md, and tidy the repo. Then walk me through each module so I can explain it.

IF YOU DRIFT: Stop. Re-read the requirements and design files. List what you changed that was not in the plan, and ask me before continuing.